

Actuarial & Life-Contingency Code Walkthrough

A maintainer's guide to the “for life” / pay-on-death feature: the integration layer present in the DOS source, the interface contract it pins down, and the reconstructed computational core.

Read §0 first. This flow is unique among the Per%Sense engines: the DOS source contains the *integration layer* but **not** the *computational core* (the ACTUARY unit and its mortality table). That core is reconstructed in the Go port. This document keeps the two strictly separated and labels every claim accordingly.

Scope. The actuarial integration sites in PRESVALU.pas, PVLXSCRN.pas, pvltable.pas; the types and runtime state in PETYPES.PAS / PEDATA.pas; the interface contract; and the Go reconstruction in internal/finance/actuarial/.

Authority & provenance. DOS line numbers cite legacy/src/dos_source/. The source-availability findings are from docs/actuarial_source_investigation.md and were re-verified against the tree.

Prerequisite. This feature is an overlay on the present-value engine; read present_value_dos_walkthrough.pdf first.

Contents

0 How to read this document — the provenance caveat

1 What is present vs absent

[1.1 The split](#) · [1.2 The missing ACTUARY unit](#) · [1.3 Why we know it shipped](#)

2 The interface contract

[2.1 The three functions](#) · [2.2 Contingency types](#) · [2.3 Runtime state](#)

3 Integration semantics (the hard-to-guess part)

[3.1 LifeProb as a weight](#) · [3.2 The JJM special case](#) · [3.3 POD add/subtract symmetry](#) · [3.4 Fancy & table](#)

4 The reconstructed core (Go)

[4.1 contingency.go](#) · [4.2 table.go](#) · [4.3 The contingency truth table](#)

5 Reconstruction audit: what the modules pin down

[5.1 Constraints recovered from the scattered code](#) · [5.2 What remains unrecoverable](#) · [5.3 The confidence cap](#)

6 Appendix: data-flow sketch & file:line index

0 How to read this document — the provenance caveat

The life-contingency feature (the manual's “Actuarial window”, used to value an annuity that continues *for life* or a benefit paid *on death*) was a real, shipped feature. But it occupies an unusual position in the materials we hold, and this document is structured around that fact.

The DOS source is half of this feature. What is present is the *integration layer* — the type system, the runtime state, and ~36 `{$ifdef ACTU}` call sites that show exactly *how* the actuarial core plugs into present value. What is **absent** is the *computational core*: a unit named ACTUARY containing the three functions `LifeProb`, `PODValue`, `XPODValue` and the underlying mortality table. Those functions are *called* everywhere but *defined* nowhere in the repository — not in any `.pas`, not in a `.dcu`, not even as strings in the shipped `Persense.exe`. The core is reconstructed in the Go port. See `docs/actuarial_source_investigation.md`.

Practically, this means:

- §1–3 (the DOS integration layer and interface contract) are **recoverable and citable** — this is solid ground, with exact `file:line` references into `dos_source`.
- §4 (the computational core) describes the **Go reconstruction**, which satisfies the DOS contract and is correct against canonical actuarial mathematics, but cannot be diffed against the original.
- §5 states precisely **what cannot be verified** and why actuarial confidence is capped below the other engines.

Throughout, DOS citations look like `PRESVALU.pas:211` and refer to `legacy/src/dos_source/`; Go citations name the file, e.g. `contingency.go:142` under `internal/finance/actuarial/`.

1 What is present vs absent

1.1 The split

Layer	Status	Where
Contingency type codes & per-row fields	PRESENT	PETYPES.PAS:169–171, 578, 596
Runtime actuarial state & init	PRESENT	PEDATA.pas:288–290, 1097
Letter mapping (actchar)	PRESENT	PEDATA.pas:144
~36 <code>{ifdef ACTU}</code> integration sites	PRESENT	PRESVALU / PVLXSCRN / pvltable
Default table names ('MALE', 'FEMALE')	PRESENT	PEDATA.pas:66
LifeProb / PODValue / XPODValue definitions	ABSENT	the ACTUARY unit (not in repo)
The mortality / life table values	ABSENT	(not in repo)
ActuarialCalc / ComputeUnknownPOD dispatch	ABSENT	(called at PRESVALU.pas:1199,1226,1268; defined nowhere)

1.2 The missing ACTUARY unit

The unit is named explicitly — but **commented out** — in the uses clauses of both consumers: `//{ifdef ACTU} ,ACTUARY {endif}` at `PRESVALU.pas:12` and `pvltable.pas:6`. A direct grep confirms every occurrence of `LifeProb`, `PODValue`, and `XPODValue` in `dos_source` is a *call*, never a function definition. The build flag `ACTU` is never `{define}`d in the snapshot (the Windows config defines `V_3`; `SCROLLS`; `PVLX`, not `ACTU`), so all 36 integration blocks are dead in *this* build — though they were live in the shipped actuarial build.

1.3 Why we know it shipped

The feature is documented in the user manual (“to calculate an annuity that continues *for life*, use the Actuarial window — see Example 23”), and the source carries the default table names `actuarialfilename`: `('MALE', 'FEMALE')` (the 1988 HHS male/female tables) `PEDATA.pas:66`, backed by an `array[1..2]` of `str8` field `PETYPES.PAS:434` whose compiled `.dcu` binary-matches. So the issue is not that the code was never written — it is that the specific ACTUARY files are missing from this snapshot. The recovery path (obtain `ACTUARY.PAS/.dcu`, or the table name/year, or the manual's worked Example 23 outputs) is detailed in the investigation doc.

2 The interface contract

Even without the core, the call sites pin down the exact signatures and semantics the core must satisfy — which is the hard-to-guess part and the part the Go port reconstructs against.

2.1 The three functions

```
function LifeProb(date: daterec; contingency: byte): real;
{ P(the contingency condition holds at `date`), given the lives' DOBs (dob[1],
  dob[2]) and the valuation date actu_now. NOT_CONTINGENT -> 1.0.
  Used as a MULTIPLICATIVE WEIGHT on each cash flow. }

function PODValue(asof: daterec; rate: real): real;
{ PV at `asof`, discounted at fixed `rate`, of the death benefit `pod` paid at
  the uncertain moment of death over the relevant life(s). }

function XPODValue: real;
{ As PODValue, but discounted through the variable-rate schedule (cc[]) to
  d^.xasof -- the PVLfancy path; reads globals, takes no args. }
```

(Signatures per docs/actuarial_source_investigation.md, corroborated by the call sites in §3.)

2.2 Contingency types

The contingency codes `PETYPES.PAS:169–171` and their screen letters `PEDATA.pas:144`. Each lump-sum row carries act0 `PETYPES.PAS:578`; each periodic row carries actn `PETYPES.PAS:596`.

Code	Const	Letter	Condition (pays if...)
0	NOT_CONTINGENT	N	always (no contingency)
1	LIVING	L	life #1 is alive
2	DEAD	D	life #1 is dead
3	ONLY_1_LIVING	1	#1 alive AND #2 dead
4	ONLY_2_LIVING	2	#2 alive AND #1 dead
5	EITHER_LIVING	E	at least one alive (last-survivor)
6	BOTH_LIVING	B	both alive (joint-life)

2.3 Runtime state

The actuarial inputs live in module globals `PEDATA.pas:288–290`: `actu_now` (valuation date), `termdate`, `dob[1..2]` (dates of birth for up to two lives), `pod` (death benefit entered by the user), `podval` (its present value), and `actset` (the set of valid contingency letters). `InitActuarial` `PEDATA.pas:1097–1111` zeroes these, sets `actu_now := now`, and (under ACTU) builds `actset` from `actchar` — and it runs *even in non-ACTU builds*, because the state is serialized to file regardless.

3 Integration semantics (the hard-to-guess part)

The master switch is the global boolean `fold_in_life` (a real variable only under ACTU; a false constant otherwise). Every integration site is gated on it. The structure is a clean symmetry: the survival weight and the death-benefit term are *added* on the forward pass and *removed* on the backward pass.

3.1 LifeProb as a multiplicative weight

Each discounted cash flow is multiplied by its survival probability. Forward, on a lump value

`PRESVALU.pas:211–213`:

```
val0 := amt0 * exp(r.rate * YearsDif(asof,date));      { discount }
if (fold_in_life) then val0 := val0 * LifeProb(date, a[i]^act0); { survival weight }
```

Backward (value→amount) the weight is *divided out*, with an underflow guard `PRESVALU.pas:219–229`:

```
p := LifeProb(date, a[i]^act0);
if (p > teeny) then amt0 := amt0 / p
else begin MessageBox('...beyond life span.');
```

A near-zero probability (the date is beyond the life span) is trapped rather than producing an infinity. The same divide-with-guard recurs in the backward solver `:874–882`. The weighted present value of a cash flow is thus $\text{payment} \cdot e^{-r \cdot \Delta t} \cdot \text{LifeProb}(t, \text{contingency})$ — a discount factor times a survival factor.

For periodic streams, switching on a survival weight forces the engine onto the **slow exact (period-by-period)** summation — the closed-form geometric series cannot carry a per-period weight `:290,389`. Each period's term is multiplied by `LifeProb(t, b[j]^actn)` `:296–297, 396–397`.

3.2 The JJM special case

The single most non-obvious line. `PRESVALU.pas:399–402`:

```
if (b[j]^actn in [DEAD,ONLY_1,ONLY_2]) and (DateComp(t,actu_now) <= 0) then part := 1;
```

The summation loop terminates when `abs(part) > teeny` fails. For death-conditioned contingencies (DEAD, ONLY_1, ONLY_2), `LifeProb` is ≈ 0 *before* `actu_now` (the person is presumed alive now, so hasn't died yet) — but the paying periods come *later*. Forcing `part := 1` for those early periods keeps the `part < teeny` test from truncating the loop before it reaches the periods that actually pay. Annotated “JJM 2/28/93” in the source. A port that drops this will silently produce a zero value for death-conditioned annuities.

3.3 POD add/subtract symmetry

The death benefit's present value participates in every evaluation. Forward, it is **added** to the grand total

`PRESVALU.pas:688–690`:

```
if (fold_in_life) then sumvalue := sumvalue + PODValue(asof, r.rate);
```

Before a backward solve, it is **subtracted** from the target first — so the residual the solver isolates is purely the cash-flow value `PRESVALU.pas:840–851`:

```
if (fold_in_life) then
  if PVLfancy then val := val - XPODValue { variable rate }
  else          val := val - PODValue(c[1]^asof, c[1]^r.rate); { fixed }
```

Conceptually the POD value is the expectation of the discounted benefit over the death-time density, $\int \text{pod} \cdot v(t) \cdot f_{\text{death}}(t) dt$ over the life(s). The exact DOS quadrature is unknown (§5); the Go core uses a discrete annual summation (§4).

3.4 Fancy & table integration

The variable-rate path mirrors the fixed one: LifeProb weights fancy lump values `PVLXSCRN.pas:248,255` and each fancy periodic term `:326`, and the fancy grand total adds the precomputed podval `:426`. In the table, PrintNextPayment splits each row's value into an “if paid” amount and a survival-weighted amount and tags the row with the contingency letter `pvltable.pas:515–519`; PrintPOD `:560–575` emits a distinct on-death line (`podval := XPODValue fancy / PODValue(...) fixed`) folded into the table total.

4 The reconstructed core (Go)

The Go reconstruction in `internal/finance/actuarial/` implements the contract of §2 against standard actuarial mathematics.

4.1 `contingency.go`

The seven contingency constants mirror `PETYPES.PAS` exactly `contingency.go:13–21`.

`ActuarialConfig :86–97` carries the two tables, the two DOBs, the valuation date `Now` (= `DOS actu_now`), the benefit `POD`, and `PODUnknown` (the back-solve flag = `DOS ComputeUnknownPOD`). `survivalProb1/2 :114–132` compute the conditional survival from `Now` to a payment date. `LifeProb :142–166` combines one or two independent lives:

$$P(\text{survive}) \text{ for each life: } s_i = \text{ConditionalSurvival}(\text{age}_i \text{ at Now, age}_i \text{ at payment})$$

`PODValue :174–178` wraps `PODValueFunc :203–238` (the variable-rate-capable analogue of `XPODValue`): a discrete annual summation of the benefit times the death-in-year probability times the discount to the year's midpoint:

$$\text{PODValue} \approx \sum_k \text{POD} \cdot [P(\text{alive at } k) - P(\text{alive at } k+1)] \cdot \text{discount}(k + \tfrac{1}{2})$$

passed through `interest.Round2`.

4.2 `table.go`

A `LifeTable table.go:30–34` holds `Qx[]/Lx[]` with radix 100,000. `SurvivalProb(age) :44–65` **linearly interpolates l_x between integer ages** (the uniform-distribution-of-deaths assumption), normalized by `lx[0]`. `ConditionalSurvival(current, future) :70–80` is the DOS-required $l_x(\text{future})/l_x(\text{current})$:

$${}_t p_x = l_{(x+t)} / l_{(x)}$$

The table is **data-driven** (loaded from CSV/JSON via `NewLifeTableFromQx/FromLx`) precisely because the original DOS table values are unknown.

4.3 The contingency truth table

With s_1, s_2 the independent survival probabilities of the two lives:

Code	Letter	Lives	LifeProb
NOT_CONTINGENT	N	none	1
LIVING	L	#1	s_1
DEAD	D	#1	$1 - s_1$
ONLY_1_LIVING	1	both	$s_1 \cdot (1 - s_2)$
ONLY_2_LIVING	2	both	$(1 - s_1) \cdot s_2$

EITHER_LIVING	E	both	$1 - (1 - s_1)(1 - s_2)$ (last-survivor)
BOTH_LIVING	B	both	$s_1 \cdot s_2$ (joint-life)

BothLiving is the joint-life status; EitherLiving the last-survivor status; Only1/Only2 the exclusive single-survivor statuses. The Go integration wiring in `presentvalue/calc.go` cites the DOS lines directly (lump weighting at `calc.go:488` → `PRESVALU.pas:212`; POD-to-total at `:549` → `PRESVALU.pas:689`), and `actuarial_canonical_test.go` validates against independent textbook oracles (pure endowment, life annuity, term insurance, two-life composition).

5 Reconstruction audit: what the modules pin down — and what they don't

The original developer's description — the actuarial code is “split across the other modules” — is accurate for the *integration* and the *annuity structure*. A second, exhaustive walk through every `{#ifdef ACTU}` site (and a search for any `DOB / age / table-reading / interpolation` code anywhere in `dos_source` and `win_source`) recovers more of the contract than the named call sites alone. None of it recovers the mortality table or the survival formula — but it meaningfully tightens what a faithful core must do, and validates the Go annuity structure.

5.1 Additional constraints recovered from the scattered code

Constraint pinned down by the modules	Evidence
Date-targeted solves are forbidden under life contingency. When <code>fold_in_life</code> is on, solving for a lump or periodic <i>date</i> raises an error (“can't perform date-targeted computations with Life Expectancy Table”) rather than iterating — because the survival weight makes value-vs-date non-invertible. A port must replicate the prohibition, not attempt the solve.	PRESVALU.pas:45, 235, 895
The life annuity is a discrete sum over scheduled payment dates. <code>fold_in_life</code> forces the engine off the closed form onto the exact period-by-period method; each rate-discounted payment is weighted by <code>LifeProb</code> evaluated <i>at that payment date</i> . So $\text{value} = \sum_t \text{payment} \cdot e^{-r \cdot (t - \text{asof})} \cdot \text{LifeProb}(t)$. This fixes the annuity discretization exactly.	PRESVALU.pas:290, 389, 397 · pvltable.pas:517–518
LifeProb is a pure multiplicative weight in [0,1]. The table prints each row's probability as <code>prob := v / ifpd</code> , where <code>ifpd</code> is the “if paid” value (discounted, <i>not</i> survival-weighted) and $v = \text{ifpd} \cdot \text{LifeProb}$. So the weight is recoverable as a ratio and is exactly the survival probability.	pvltable.pas:517–518, 533
POD participates in the rate root-find. Inside the unknown-rate secant loop (PV-8), <code>sum := sum + PODValue(asof, r.rate)</code> . So <code>PODValue</code> must be a smooth function of rate, evaluable at every trial rate — not a precomputed constant.	PRESVALU.pas:712, 790
The unknown-POD solve is structurally reconstructable. <code>podunk</code> is set when all cash-flow blanks are filled but a target value is given (“proceed only to compute an unknown POD amount”); <code>ComputeUnknownPOD</code> then back-solves the benefit. Because a death benefit's present value is linear in the benefit amount <code>pod</code> , the solve reduces to $\text{pod} = (\text{target} - \text{cash-flow value}) / \text{unit-POD-value}$. The structure is known; only the unit value (the quadrature) is not.	PRESVALU.pas:1179, 1207, 1268
	pvltable.pas:561–568

The POD is a single scalar benefit paid with certainty in the on-death row (printed at probability 1.0), valued by <code>PODValue</code> (fixed rate) or <code>XPODValue</code> (rate schedule).	
Forward/backward POD symmetry and the master switch. <code>fold_in_life</code> gates everything; POD is added forward (§3.3) and peeled before any backward solve; <code>termdate</code> (a term-certain cutoff) and <code>deathdate</code> exist as model fields.	<code>PEDATA.pas:288–292</code>

Net gain from the second pass. The recovered constraints fully determine the *life-annuity* arithmetic (a discrete, per-payment-date survival-weighted sum) and the *shape* of the POD handling (scalar benefit, linear in amount, smooth in rate, peeled symmetrically). The Go port's per-payment summation matches the recovered annuity structure. What is *not* recovered is everything inside the survival probability itself.

5.2 What remains genuinely unrecoverable

The second pass also closed off the most tempting false lead and confirmed the hard boundary:

- **The mortality table values** (MALE/FEMALE, 1988 HHS) appear only as default *file names* `PEDATA.pas:66`; no table data and no table-reading code exist in any module.
- **The survival formula and intra-year interpolation / age convention.** There is no DOB-entry, age-computation, or table-lookup code anywhere in `dos_source` or `win_source` — `dob[1..2]`, `actu_now`, and `termdate` are only declared and zeroed. The age convention (nearest- vs last-birthday) and the intra-year rule are therefore not determinable from the present source.
- **The shared LinearInterpolation/QuadraticInterpolation are *not* the table interpolators.** A natural guess is that ACTUARY reused these for $1x$ interpolation. It did not: both are *root-finders* (“what is your next guess x to make $y=0$?”) used by the iterative solvers, with no connection to mortality tables `INTSUTIL.pas:449, 458`. This lead is a dead end — recorded so no one re-chases it.
- **The PODValue death-time quadrature.** The on-death row is a single opaque scalar from `PODValue`; its internal integration over the death-time density is hidden in ACTUARY.
- **The bodies of ActuarialCalc, ComputeUnknownPOD, LifeProb, PODValue, XPODValue** — called throughout, defined nowhere (§1.2).

5.3 The confidence cap

Confidence cap. The reconstruction is correct-by-construction against canonical actuarial mathematics, and the second pass shows it satisfies a *tighter* recovered contract than first documented (notably the exact per-payment-date annuity weighting). But it still cannot be shown bit-identical to the original, because the table values, the survival/age convention, and the POD quadrature are absent. Actuarial confidence remains capped (~86 on a “faithful-to-original” basis). Obtaining any one of ACTUARY.PAS/.dcu, the table name/year, or the manual's Example 23 outputs would lift it; the .dcu alone would allow a bit-identical oracle. See `docs/actuarial_source_investigation.md` §“Recommendations.”

6 Appendix: data-flow sketch & file:line index

```

FORWARD (build value):
  lump i:      val0 = amt0·e^{r·Δyears(asof,date)}
               if fold_in_life: val0 *= LifeProb(date, act0)      PRESVALU.pas:212
  periodic j:  (EXACT method forced when fold_in_life)           PRESVALU.pas:290
               per period t: part = amt·e^{-r·Δyears(t,asof)}·cola
               if fold_in_life: part *= LifeProb(t, actn)        :297
               [JJM guard: DEAD/ONLY_1/ONLY_2 before now → part:=1] :399
  grand total: if fold_in_life: sumvalue += PODValue(asof, rate)  PRESVALU.pas:689
               (= XPODValue, fancy path)

BACKWARD (isolate the unknown):
  val = sumvalue
  if fold_in_life: val -= PODValue(asof,rate) (peel POD first)    PRESVALU.pas:843
  val -= (each known row's val0 / valn)
  solve the unknown row, dividing OUT LifeProb (guarded p>teeny) PRESVALU.pas:875

Reconstructed core (Go):  LifeProb / PODValue      actuarial/contingency.go:142 / 203
                          LifeTable lx interpolation  actuarial/table.go:44 / 70

```

Symbol	Reference
Contingency codes / per-row fields	PETYPES.PAS:169–171 / 578, 596
actchar letter map / default table names	PEDATA.pas:144 / 66
Runtime state / InitActuarial	PEDATA.pas:288–290 / 1097
Commented-out uses ACTUARY	PRESVALU.pas:12 · pvltable.pas:6
LifeProb weight (lump fwd / bwd)	PRESVALU.pas:212 / 219–229
LifeProb weight (periodic) / JJM guard	PRESVALU.pas:297 / 399
POD add / subtract	PRESVALU.pas:689 / 843
Fancy weighting / PrintPOD	PVLXSCRN.pas:248,326,426 · pvltable.pas:560
Go core (contingency / table)	actuarial/contingency.go:142 · table.go:44
Investigation report	docs/actuarial_source_investigation.md

Per%Sense DOS→Go port. DOS line numbers cite `legacy/src/dos_source/`; Go citations are under `internal/finance/actuarial/`. This feature is an overlay on the present-value engine — see `present_value_dos_walkthrough.pdf`.

Source-availability findings: `docs/actuarial_source_investigation.md`.